

Software Project Management – IS212

Software Development Processes

Software Development Processes

Objectives

On completing this module, you will be able to:

- Summarise the key **traditional** and **agile software development** processes
- Identify the **pros** and **cons** of each
- Explain where **AI-assisted** and **agentic development** are reshaping these processes

Traditional

- Waterfall
- RUP

Agile

- Extreme Programming
- Kanban
- Scrum

1960/70s – Software Crisis



*The major cause of the software crisis is that the machines have become **several orders of magnitude more powerful!***

*To put it quite bluntly: **as long as there were no machines, programming was no problem at all**; when we had a few weak computers, programming became a mild problem, and **now we have gigantic computers, programming has become an equally gigantic problem.***

– Edsger Dijkstra

**projects running
over time**

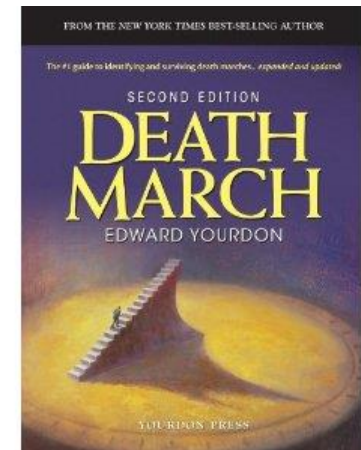
**projects running
over budget**

**software often didn't
meet requirements**

Computing power was outpacing the ability of software developers to effectively use it

**code that was difficult
to maintain**

**software never
being delivered**





*Software requirements may not be
'frozen' before development*

Software can be subject to (frequent) change

How does engineering
software compare to, say,
engineering a **bridge**?

Software and Work In Progress (WIP) are “invisible”

*Engineers increasingly supervise AI-generated
code rather than writing every line themselves*

Technical debt is a major challenge

Stakeholders can be on different wavelengths...



How the customer explained it



How the project leader understood it



How the analyst designed it



How the programmer wrote it



What the beta testers received



How the business consultant described it



How the project was documented



What operations installed



How the customer was billed



How it was supported



What marketing advertised



What the customer really needed

Software Development Processes

- Structured **software development processes** have been established to improve quality, efficiency, and predictability in software projects
- We will study five key processes (focusing on **scrum** in subsequent weeks)

Traditional

- Waterfall
- RUP

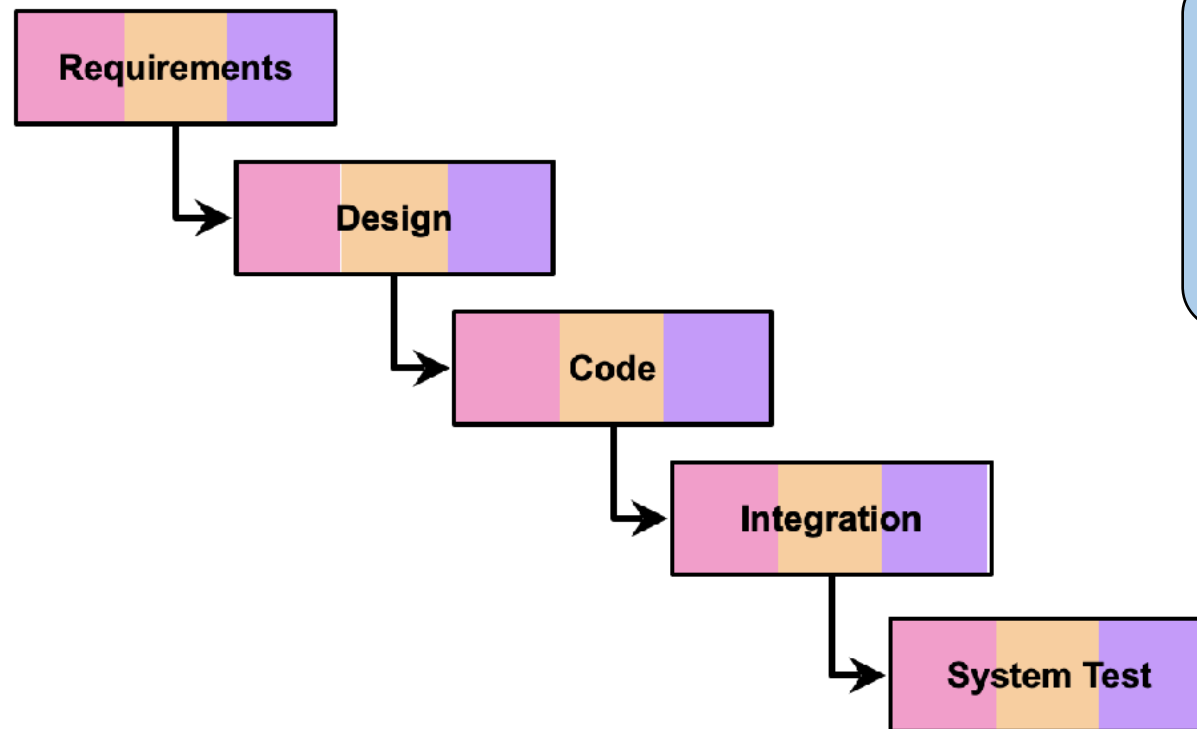
Agile

- Extreme Programming
- Kanban
- Scrum



Waterfall

Idea: break software development into linear, sequential phases



Simple, intuitive, and appealing?

When might it work well? ☒

When might it not? ☐

Waterfall – Discussion

- **Don't knock it!** Waterfall is extensively used and often works well –
 - If the project is of small to medium-scope (relatively short duration required)
 - If requirements are contractually agreed before development (e.g. when outsourcing to vendors)
 - If the product definition is stable and unlikely to evolve over time
 - If the industry is **regulated** (strict compliance with standards in healthcare, aerospace, defence, ...)
- However – waterfall's **inflexible partitioning of phases** makes it less appropriate for projects that involve a lot of **uncertainty**
- Harder to respond to **changing customer requirements**
- **Difficult to experiment**, which may stifle the team's innovation
- **Testing only at the end** of the process invites **failure**. **(Why?)**

Waterfall – Is it Making a Comeback?

- We just saw waterfall struggles with uncertainty... but **AI is prompting a second look**
- AI-assisted delivery rewards clear, **up-front specs** — vague requirements make agents produce **wrong output at scale**
- “**Spec-driven development**”: write the spec first, then let the agent build (GitHub Spec Kit, AWS Kiro)
 - *Critics ask: is this just waterfall wearing a trendy new hat?*
- Regulated work (healthcare, aerospace, finance) **never left waterfall** anyway



 Spec Kit



Up-front planning has enduring value – but evolving requirements and reviewing AI output hasn't gone away

Alternative – Develop Software Iteratively

- Iterative processes attempt to mitigate the rigidity of the waterfall process
- More flexibility
- Enhanced stakeholder collaboration
- Early detection of issues
- Development is broken down into smaller, manageable increments



Image source: <https://www.pacific-research.com/iterative-product-development/>

Software Development Processes

Traditional

iterative traditional
method →

- Waterfall
- RUP

Agile

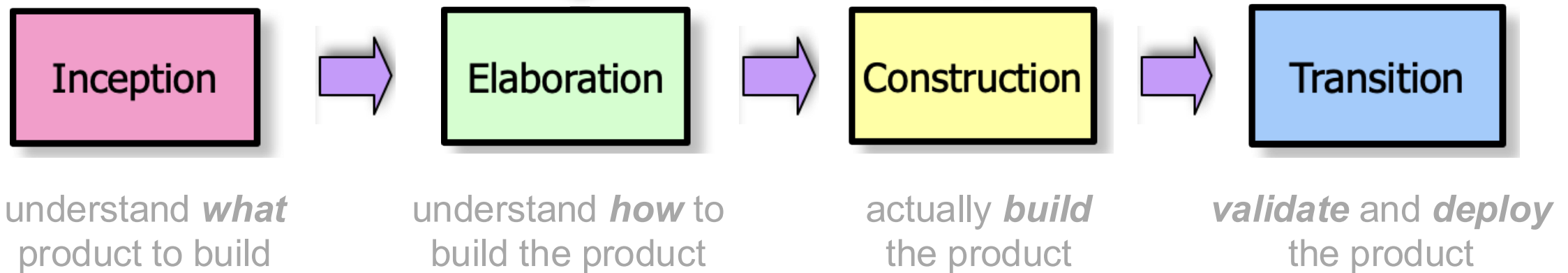
iterative agile
methods →

- Extreme Programming
- Kanban
- Scrum

Rational Unified Process (RUP)



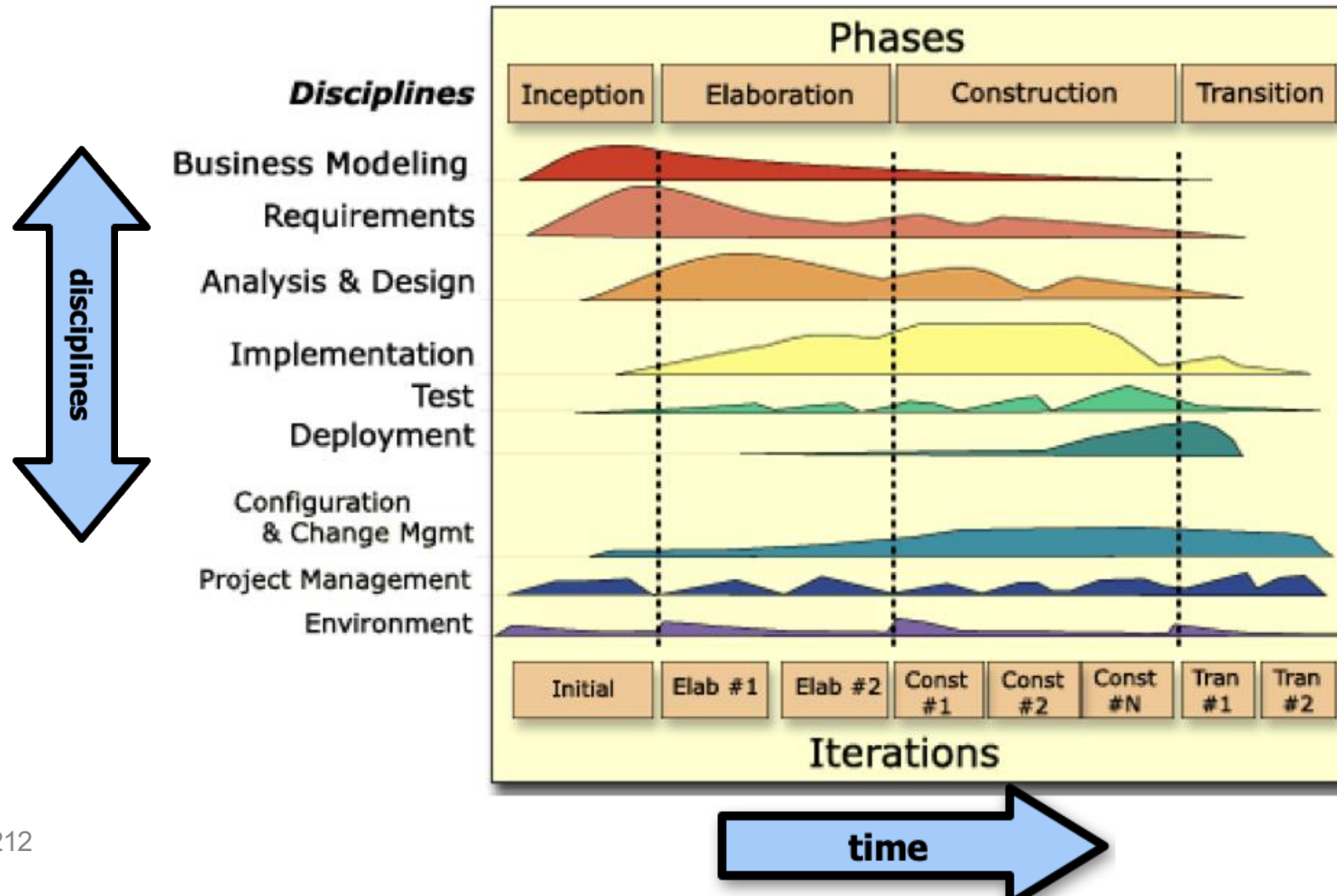
- The Rational Unified Process (RUP) consists of four phases
 - i.e. key 'milestones' for the project, similar to waterfall



- Various 'engineering disciplines' then cut across all four phases
 - e.g. business modelling, requirements, implementation, testing

Rational Unified Process (RUP)

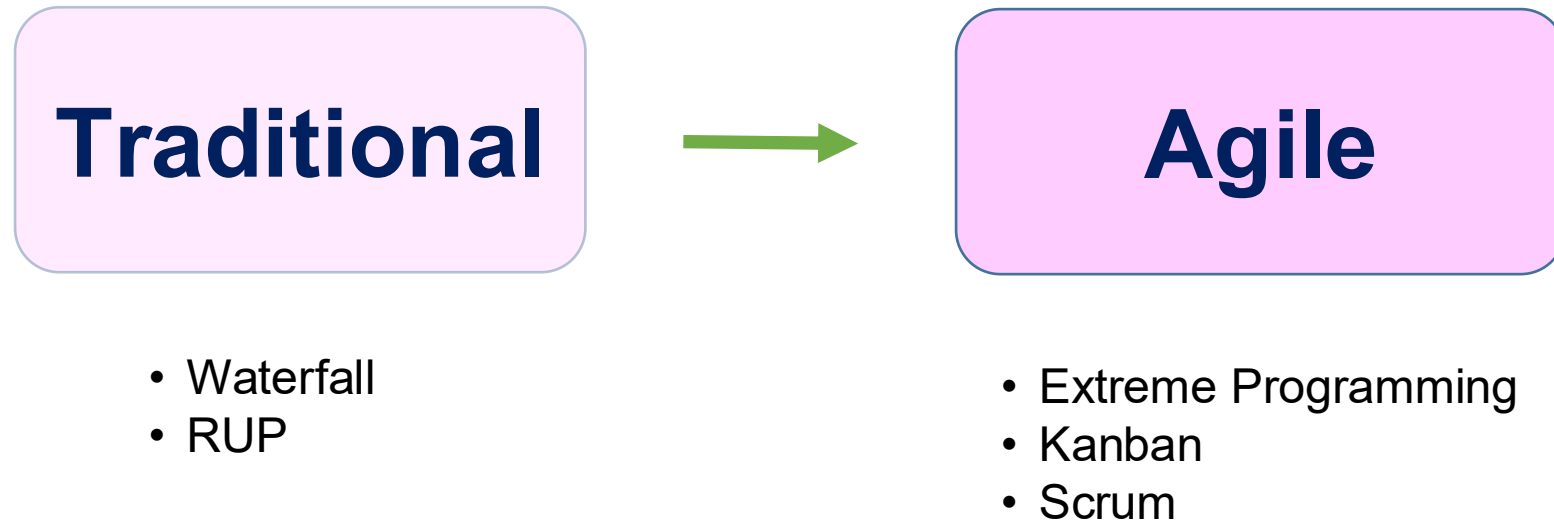
- Unlike waterfall – it **explicitly encourages iterations** within phases



RUP – Discussion

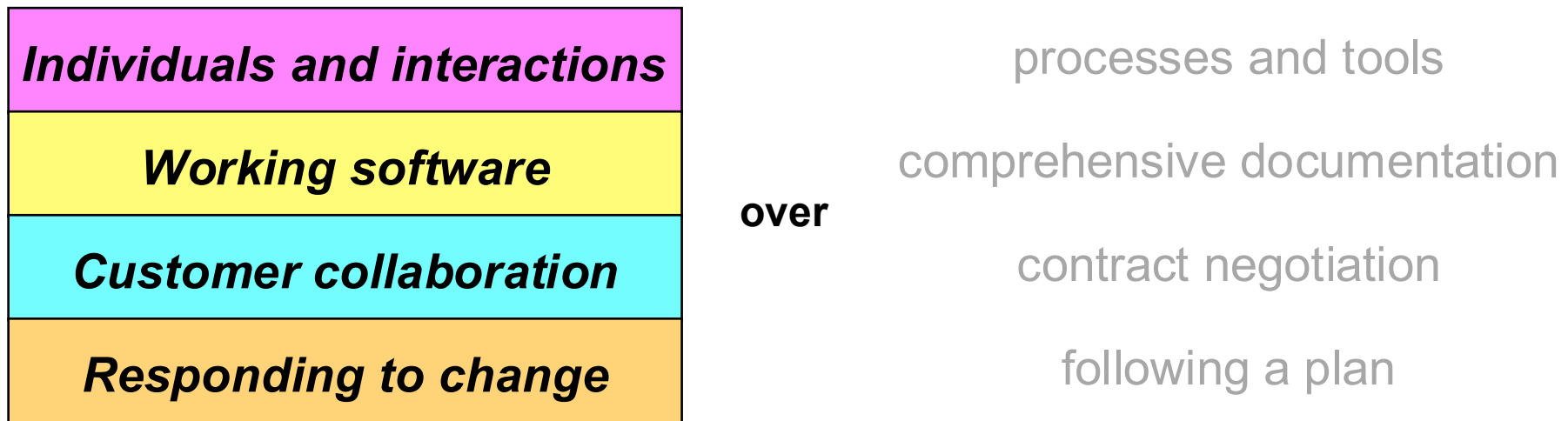
- Each phase targets a **milestone outcome** — vision, architecture, working software, deployment
- Support for **iteration** a big advantage over classic waterfall
- **Forces integration** to happen throughout the software development
- Testing can happen as early as ‘inception’ – **not left until the end!**
 - What kind of testing activities could occur before a system is built?
- **But** – RUP is **quite complex**; a lot of process ‘overhead’
 - Less flexible than agile processes (*see later!*)

Software Development Processes

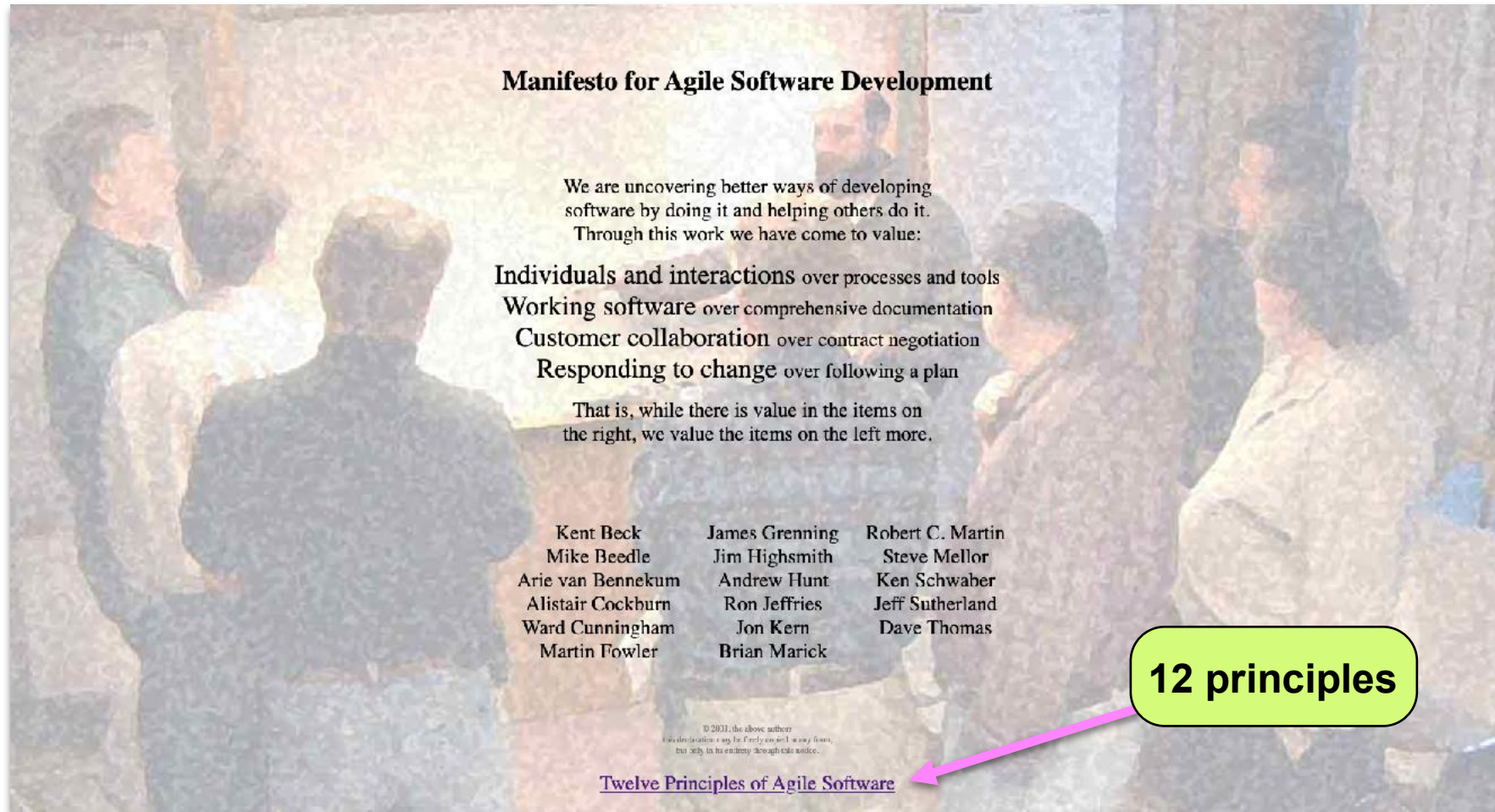


Agile – Motivation

- Traditional software processes can be overly regulated, planned, and micromanaged, i.e. **bureaucratic**
- In reaction, **lightweight software processes** started evolving that collectively became known as “**agile**”, emphasising:



Agile Manifesto (2001)



<https://agilemanifesto.org/>

Agile Principles *(re-organised; Meyer '14)*

Agile principles

Organizational

- 1 Put the customer at the center.
- 2 Let the team self-organize.
- 3 Work at a sustainable pace.
- 4 Develop minimal software:
 - 4.1 Produce minimal functionality.
 - 4.2 Produce only the product requested.
 - 4.3 Develop only code and tests.
- 5 Accept change.

Technical

- 6 Develop iteratively:
 - 6.1 Produce frequent working iterations.
 - 6.2 Freeze requirements during iterations.
- 7 Treat tests as a key resource:
 - 7.1 Do not start any new development until all tests pass.
 - 7.2 Test first.
- 8 Express requirements through scenarios.

key insight



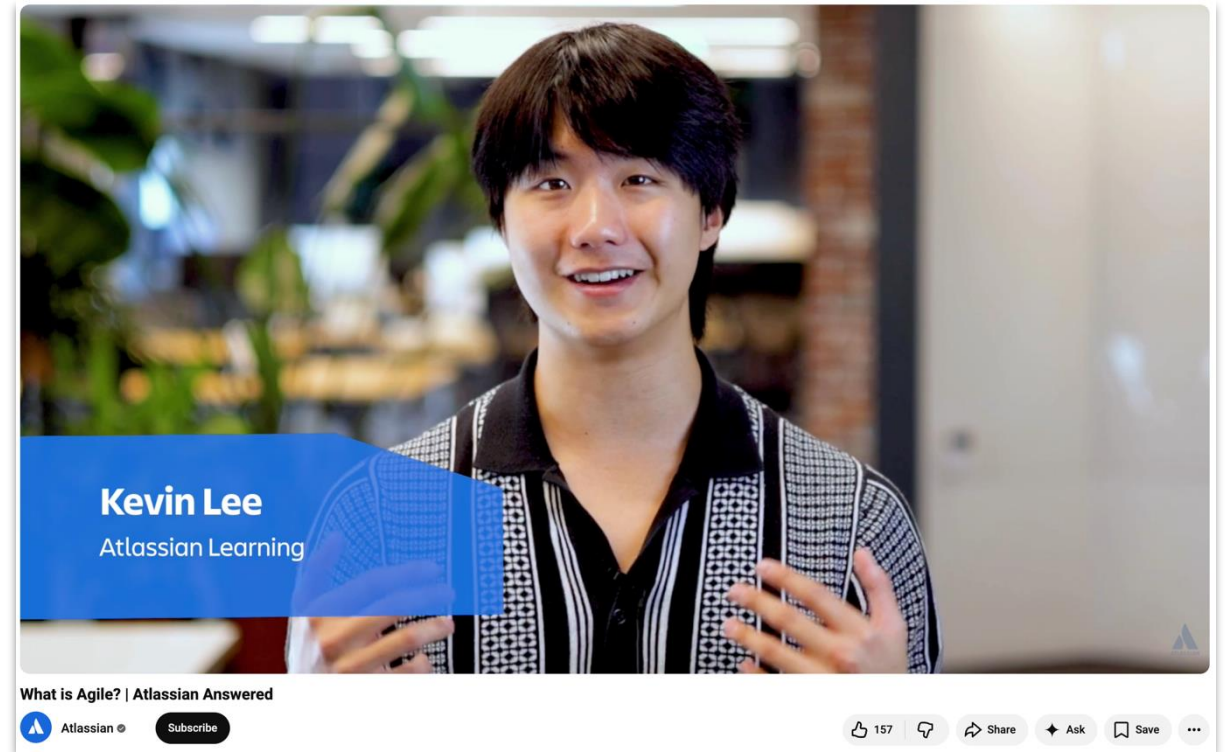
Bertrand Meyer

Atlassian – What is Agile?

Padlet Exercise

Lightbulb (💡): Share one thing from the video that makes sense and seems quite useful

Speedbump (⚠️): Share one doubt, hesitation, or question you have about how this works in the real world



<https://www.youtube.com/watch?v=4Ugsu2n2cLE>

Highly iterative and flexible

Working software delivered frequently

Working software (not “WIP”) is the measure of progress

Agile – Key Characteristics

Late changes in requirements – no problem

Close, daily co-operation

Visualise “WIP” to manage team’s capacity and highlight constraints

Software Development Processes

Traditional

- Waterfall
- RUP

Agile

- Extreme Programming
- Kanban
- Scrum

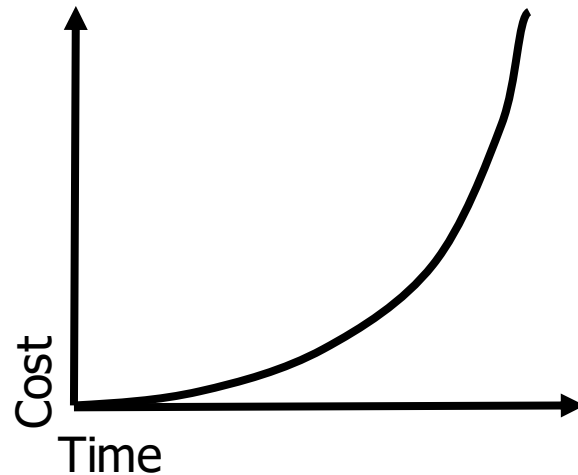


Extreme Programming (XP)

- Premise of **extreme programming (XP)**

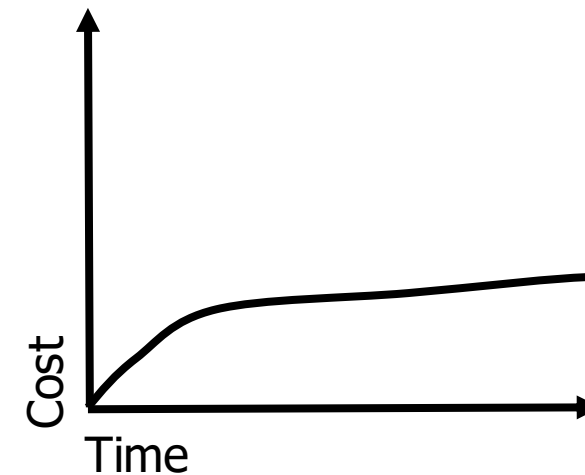
Traditional

- **upfront** design
- code late
- release when 'done'



XP

- **continuous** design
- code early
- release early



Extreme Programming (XP)

- eXtreme programming (XP) advocates frequent releases in short development cycles
- **Idea** – take good software development practices to a logical extreme
- Code reviews are beneficial...
*...so review code continuously (**pair programming**)*
- Automated testing is beneficial...
*...so write your tests before you write your code (**Test-Driven Development**)*

Question – how is agentic coding changing practices like pair programming and test-driven development?

Extreme Programming (XP)

- XP practices are based on **four principles**

Coding is the core activity

Code doesn't only deliver the solution
– use it to explore / explain problems too!

LOTS of testing during dev

Design unit tests or software contracts first
– *then* code! (aka **Test-Driven Development**)

Developers and customers communicate directly

Programmer must understand the business requirements to design a technical solution

Regular refactoring

Overall system design is considered during regular **refactoring** exercises

Fine-scale feedback

- Pair programming
- Planning game
- Test-Driven Development
- Whole team

Shared understanding

- Coding standard
- Collective code ownership
- Simple design
- System metaphor

12 XP Practices

Continuous process

- Continuous integration
- Design refactoring
- Small releases

Programmer welfare

- Sustainable pace

XP – Impact

- XP provided a ‘jolt’ that brought attention to agile methods ⚡
- Some see XP as ‘dogmatic’; others see it as a consistent, strong view of how programming should be practiced
- Many of the individual practices promoted by XP have left their mark on industry, especially that:
 - Projects should integrate code all the time
 - Tests are a key resource, and should be run against code often
- XP practices like test-driven development can be integrated with other software development processes (e.g. scrum)
- Some XP practices have gained new prominence in AI era, e.g. CI/CD pipelines (GitHub Actions) as guardrails for AI-generated code quality

Software Development Processes

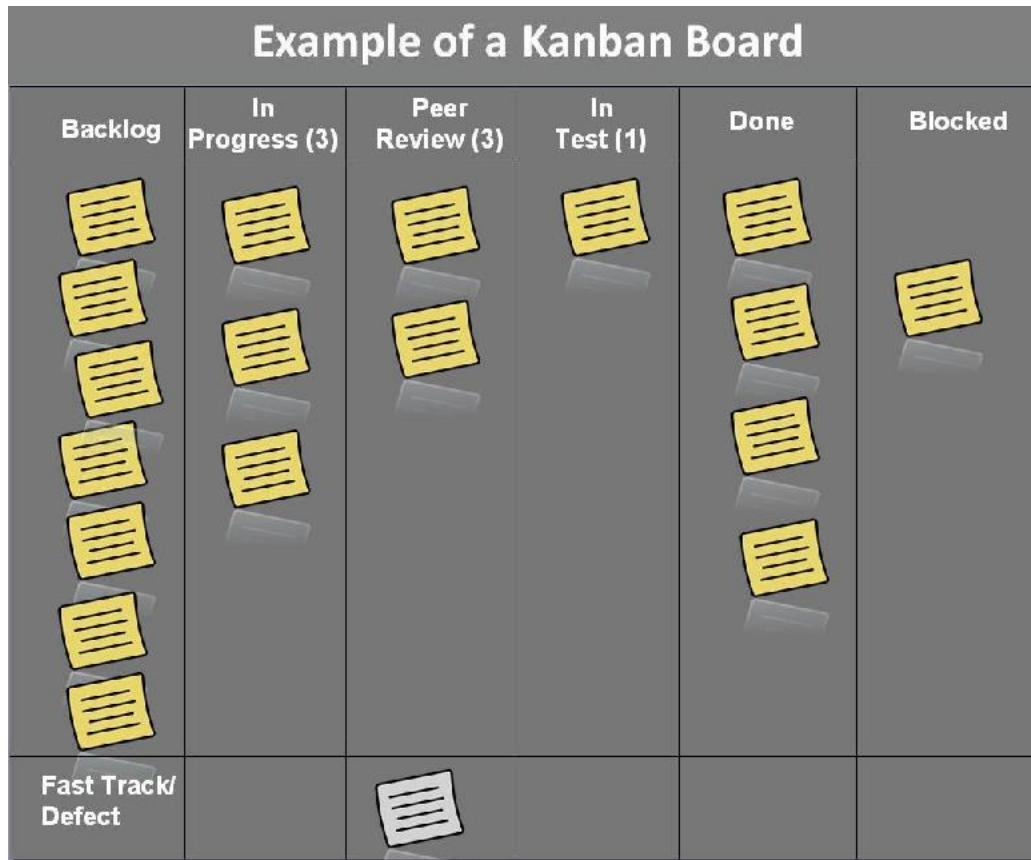
Traditional

- Waterfall
- RUP

Agile

- Extreme Programming
- Kanban
- Scrum

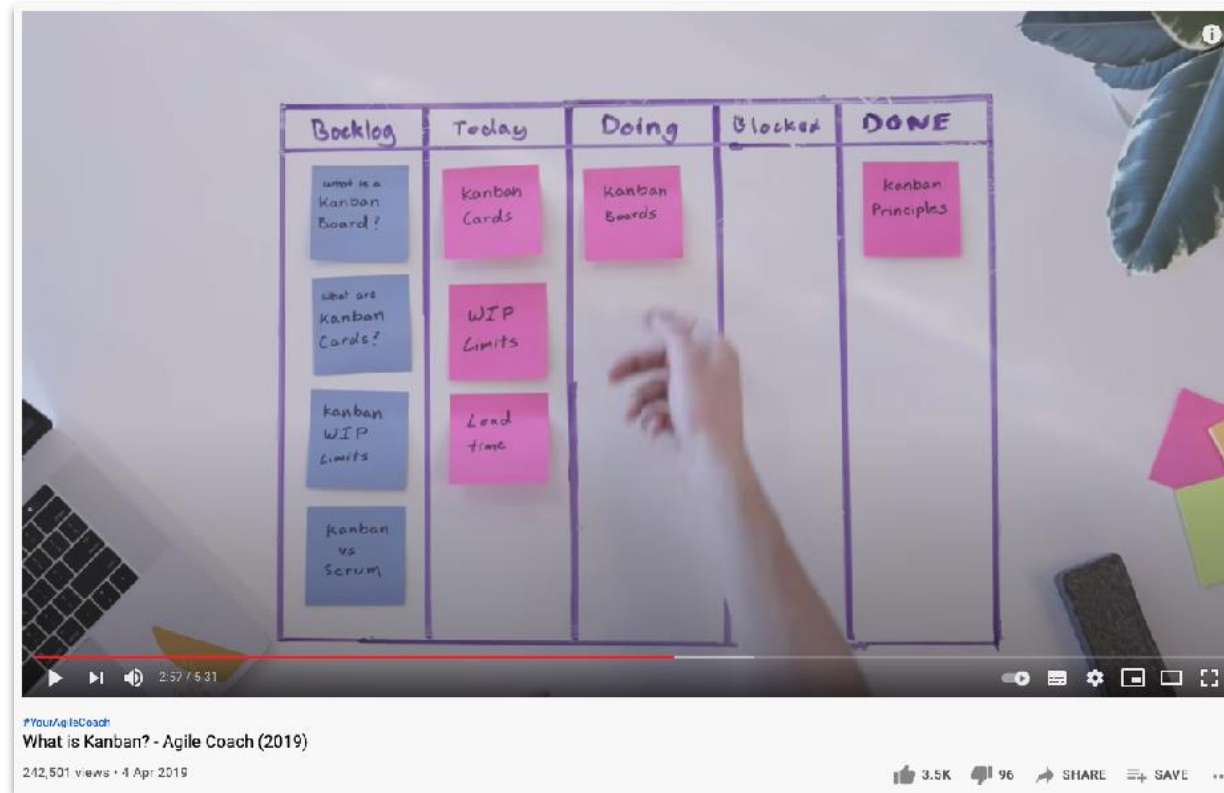
Kanban



“Kanban”

- Japanese word meaning (roughly) “large visual board”
- *idea*: visualising a team’s WIP helps to track workflow and identify constraints
- NB: in the Kanban process, a single board is used **continuously** throughout the *whole project*

Kanban

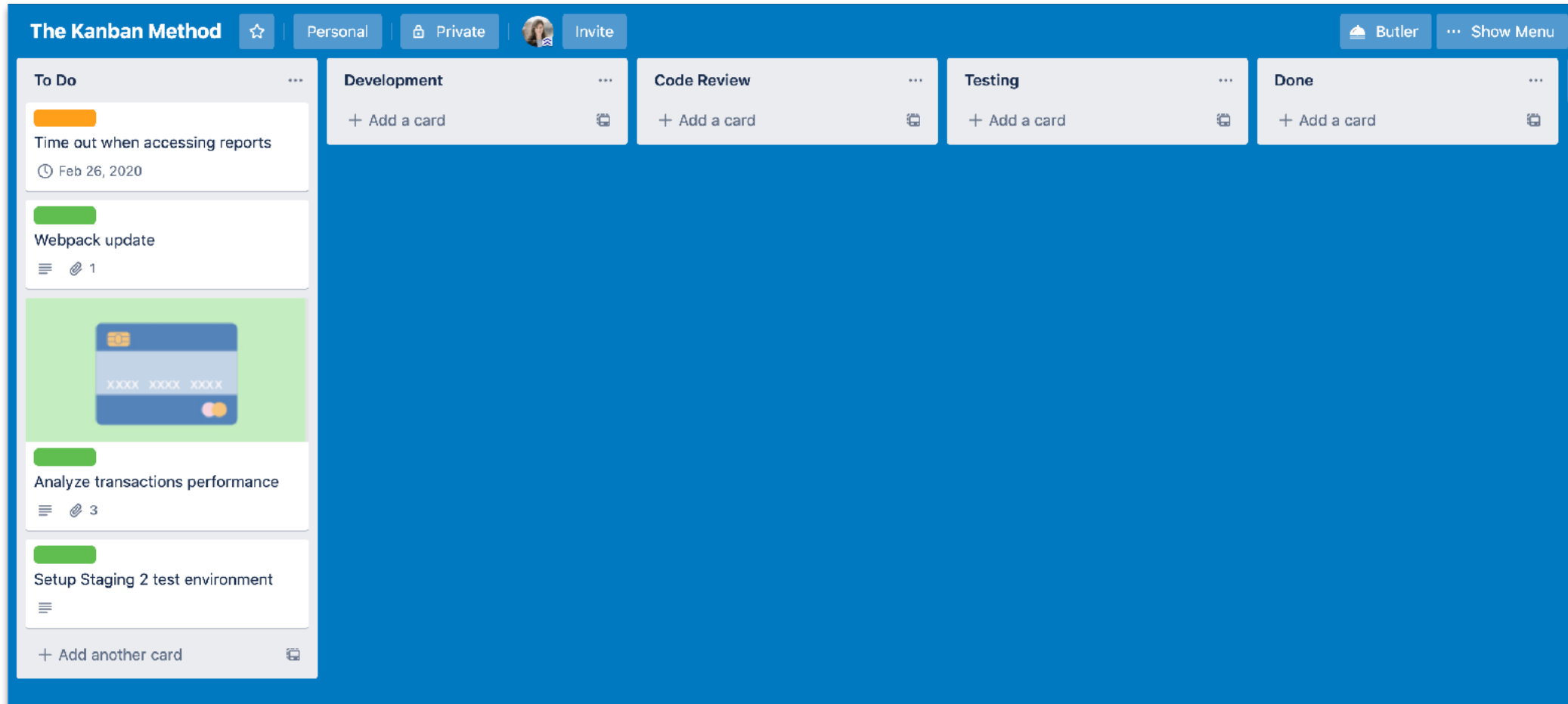


<https://www.youtube.com/watch?v=iVaFVa7HYj4&t=116s>

Kanban

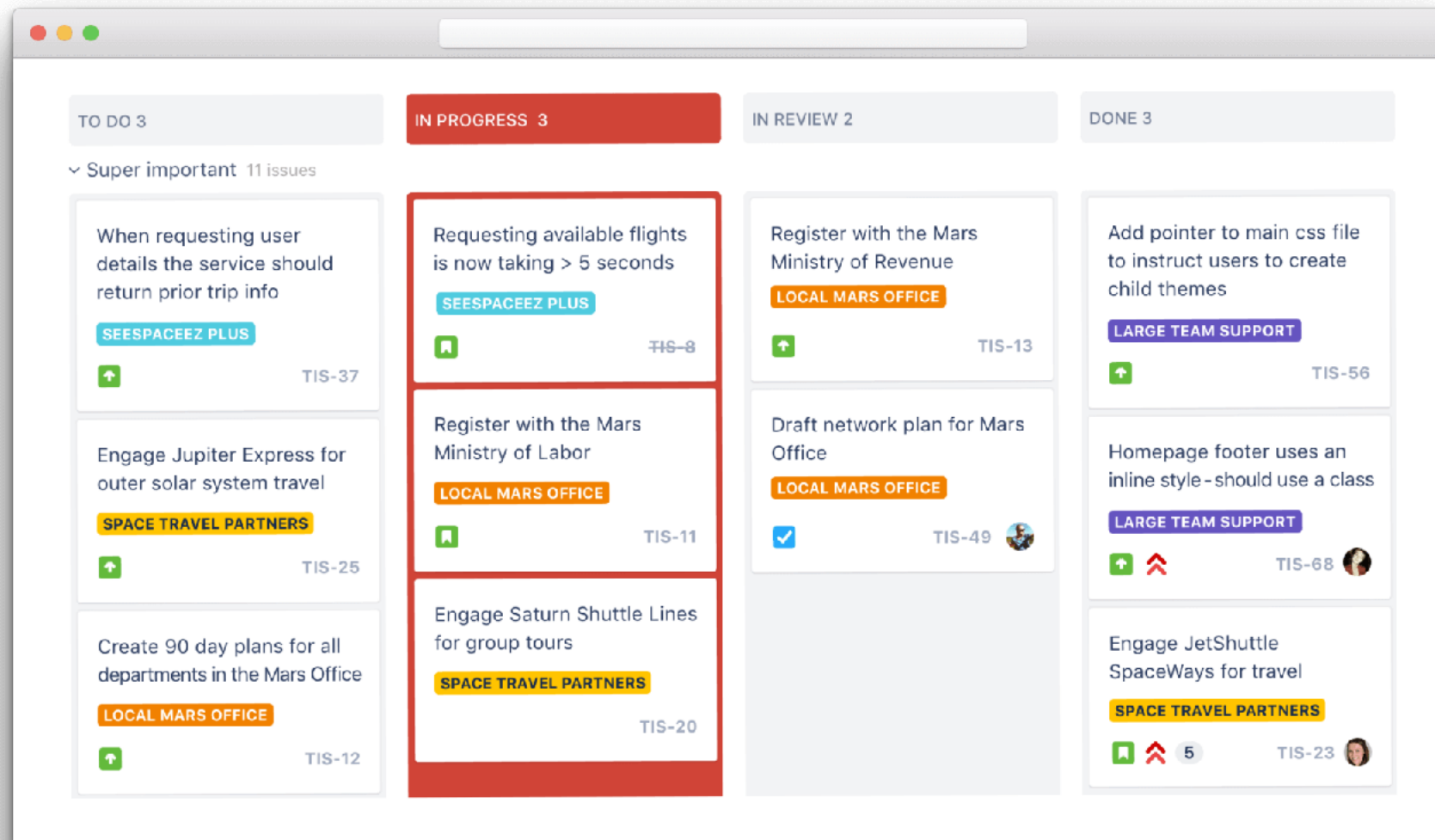
- Work-in-progress (WIP) doesn't deliver value until deployed
- Limiting / minimising WIP makes it easier to identify inefficiencies and constraints in a team's workflow
- Kanban boards clearly visualise WIP and match to team's capacity
- Simple, but effective approach to help teams coordinate

Kanban – Virtual Boards



<https://trello.com/en>

Kanban – Virtual Boards



Software Development Processes

Traditional

- Waterfall
- RUP

Agile

- Extreme Programming
- Kanban
- Scrum

Scrum



“scrum”

- *borrowed from rugby
(formation of players)*

- *term introduced to
emphasise **teamwork***

We will follow the **scrum process**
for the IS212 project

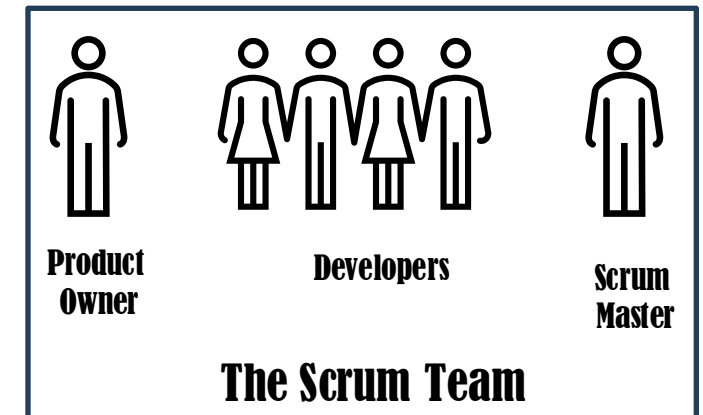
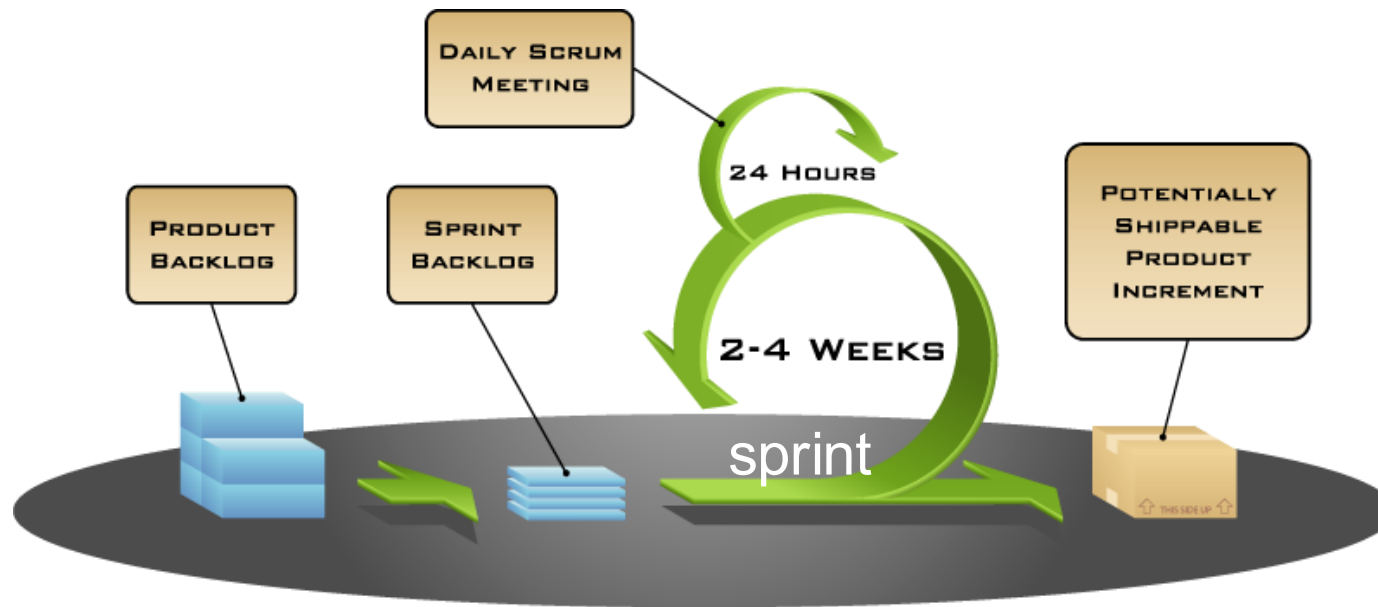
Scrum



<https://www.youtube.com/watch?v=TRcReyRYIMg>

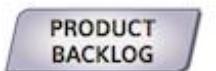
Scrum

- **Scrum** is a **lightweight, iterative framework** for developing software
- The iterations are structured “**sprints**”, which are designed to foster collaboration, accountability, and continuous improvement



What is a Product Backlog?

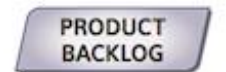
- A **product backlog** is a **prioritised list** of features that:
 - Will deliver an overall **Product Goal**
 - Adds **Value** to the customer
 - Is **Estimated** (i.e., amount of effort required)
- It mainly contains items that bring **direct value** to customers
 - Features expressed as **User Stories** (see **Week 2**)
- It can also contain items that bring **indirect value** to customers
 - Enhancements and fixes
 - Infrastructure set up (e.g. Git repository, CI/CD pipeline)
 - Exploratory research work
- It does **not** contain **low-level tasks**
 - e.g. “send email to Rajesh” ❌, “implement PersonDAO class” ❌
- The **Product Owner** is accountable for it; refinement is collaborative
- It is a **living document** that continuously evolves throughout the project



Product Backlog Example (More in Weeks 2–3)

Product Goal: To provide a convenient way for shoppers to purchase products online

Priority	User Story	Estimate	More Details (Jira Link)	Status
1	As a shopper, I want to browse the catalog of products, so that I know what to buy	3	https://shopmore.atlassian.net/browse/JIRA-10	Done (Sprint 1)
2	As a shopper I want to add a product to my cart, so that I can buy the product	8	https://shopmore.atlassian.net/browse/JIRA-23	Done (Sprint 1)
3	As a shopper, I want to remove a product from my cart so that I don't buy the product	5	https://shopmore.atlassian.net/browse/JIRA-15	In-Prog (Sprint 2)
4	As a shopper, I want to checkout my cart so that I complete my shopping	13	https://shopmore.atlassian.net/browse/JIRA-09	In-Prog (Sprint 2)
7	As a shopper I want to choose payment on delivery, so that can pay after receiving my product	1	https://shopmore.atlassian.net/browse/JIRA-14	To Do



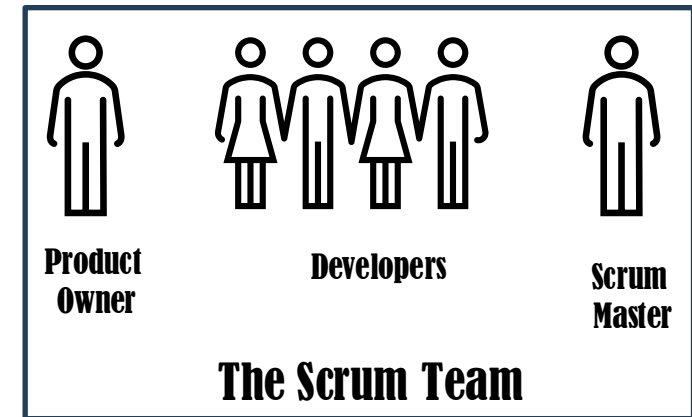
Scrum – Team Members (“Accountabilities”)

Developers

- Design, code, test, and integrate the software product
- Deliver high-quality shippable software increments
- Self-organised and cross-functional
- Pace development in a sustainable manner

Scrum Master

- Facilitates scrum meetings (standups, reviews, retros)
- Removes impediments / blockers
- Facilitates mentorship and conflict resolution
- In small teams, may also take on development tasks

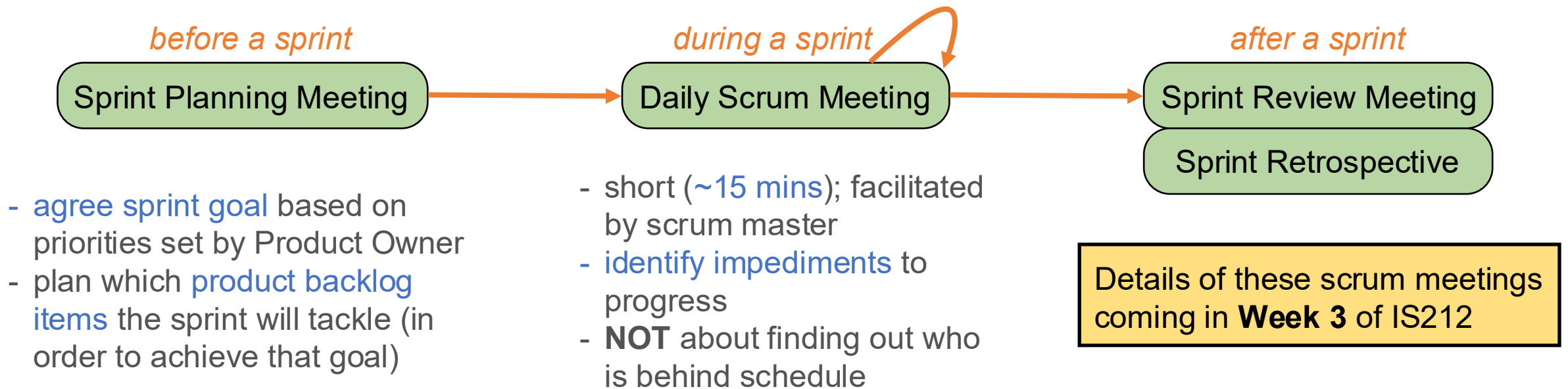


Product Owner

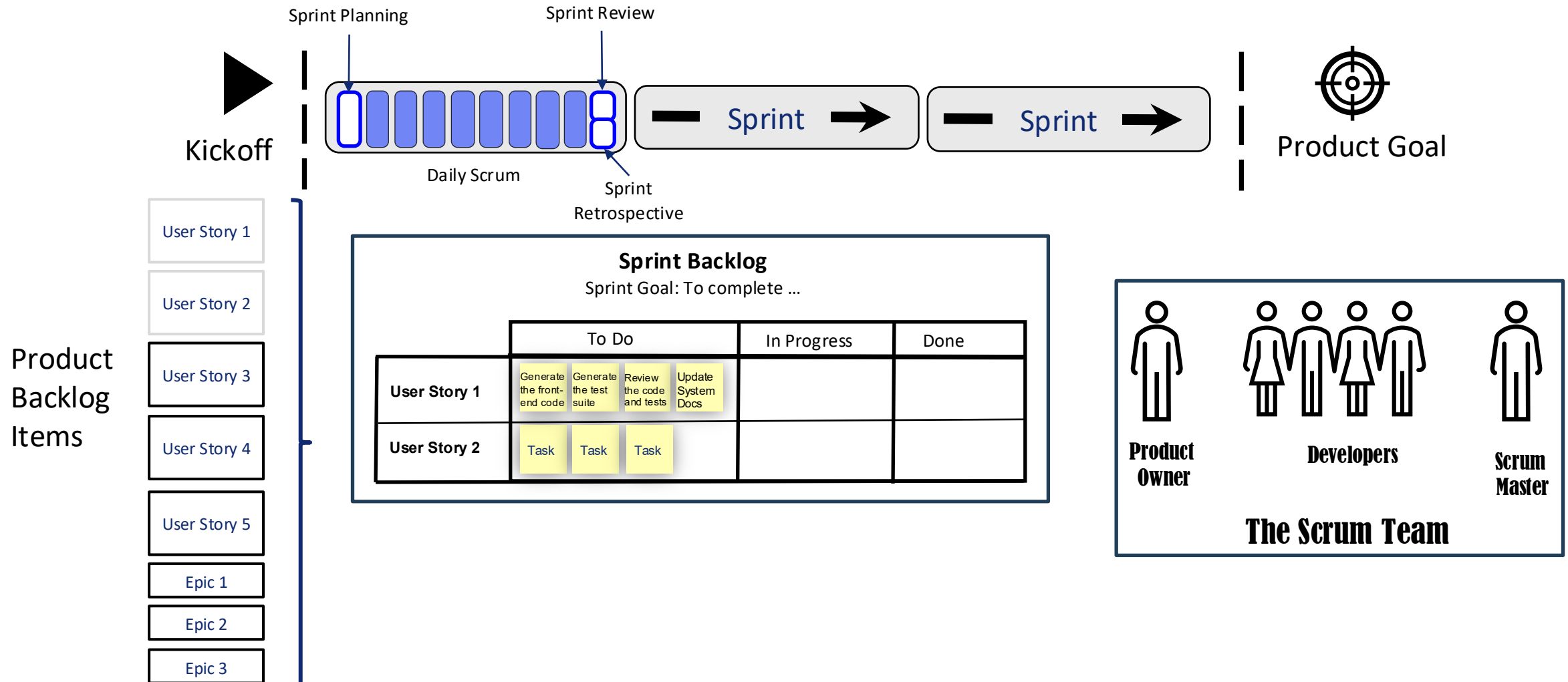
- Identifies and (re-)prioritises product features
- Writes user stories and ‘grooms’ the product backlog
- Collaborates with stakeholders (e.g. the customer)
- Validates product increments after each sprint

Scrum – Sprints

- **Sprints** are fixed-length (e.g. 2, 3, or 4 week) iterations that aim to achieve an agreed **sprint goal**, e.g. a particular software increment
 - For the IS212 project, we recommend choosing a sprint length of **1 or 2 weeks**
 - **Sprint goal is frozen** for the length of a sprint
- Sprints are lightly managed using “**time-boxed**” scrum meetings



Scrum – Overall Picture (More in Weeks 2–3)



Sprint Backlog

- The **sprint backlog** lists the product backlog items (e.g. user stories) to be **completed in the current sprint**
- The **sprint goal** is typically “frozen”  for the duration of the sprint. **(Why?)**
- The stories are associated with one or more implementation **tasks**
 - *E.g. set up database, generate front-end code, review & validate code, ...*
- The tasks are placed on a **scrum taskboard** to visualise progress
 - *Updated by team during stand-up meetings*
 - *Monitored by scrum master – **stagnant WIP** a warning sign!*

Sprint Backlog						
Sprint Goal: To complete ...						
		To Do			In Progress	Done
User Story 1		Generate the front-end code	Generate the test suite	Review the code and tests	Update System Docs	
User Story 2		Task	Task	Task		

How to get Scrum right

- You will use the scrum process for the **IS212 project** (starting from **Week 4**)
- You are encouraged to **learn by doing** – get your hands dirty, make mistakes in sprint #1, reflect, and then improve
- We shall ‘deep dive’ into the scrum process in **Weeks 2–3**
- As we make this journey together, please set aside some time for the required **complementary reading** in eLearn:
 - *Scrum Guide & Scrum Primer*
 - *(New for AI) 2025 Scrum Expansion Pack:*
<https://scrumexpansion.org/>

Ken Schwaber & Jeff Sutherland

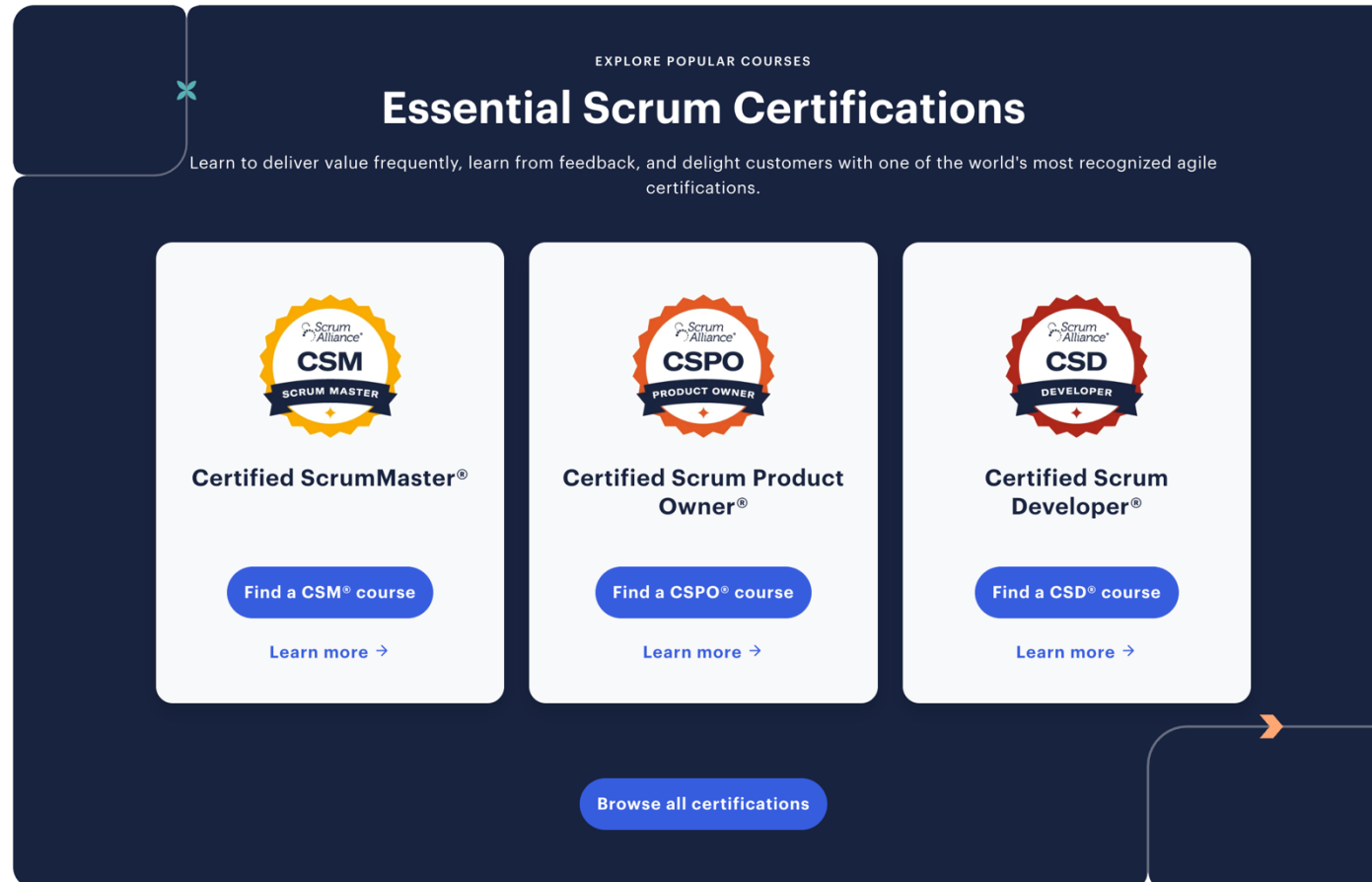
The Scrum Guide

The Definitive Guide to Scrum: The Rules of the Game



Scrum – Certification

- The **Scrum Alliance** provides a number of **certification routes** for those keen to develop their agile credentials



<https://www.scrumalliance.org/>

Scrum – Impact

- Scrum has turned iterative development into a **precise discipline**
 - *Codified goals, duration, and management of iterations (sprints)*
- **Savvy marketing** operation – certifications turn scrum learners into scrum supporters
- Sprint-based ways of working are now common across a wide range of industries, extending **well beyond software development**
- IS212 teaches **one interpretation** of the scrum rules. In practice, many real-world teams may ‘tweak’ them in various ways
 - *Or adapt scrum ideas into other processes, e.g. **daily standups in waterfall***

Scrum vs. Kanban – similar boards, but **different**

- **Scrum** is a different process to **Kanban** despite the similar boards 

	Scrum	Kanban
Cadence	Fixed-length sprints	Continuous flow
Delivery	Increment produced each Sprint; release can occur anytime	Continuous / on-demand delivery
Accountabilities / Roles	Product Owner, Scrum Master, Developers	No prescribed roles
Key metrics	Often velocity; progress toward Sprint/Product Goals	Lead time, delivery rate/throughput, WIP
Change philosophy	Sprint Goal protected; scope may be renegotiated	Work can be reprioritised as capacity becomes available
Visualisation	Scrum board cleared after every sprint	Kanban board used throughout project

Software Development Processes

Traditional

- Waterfall
- RUP

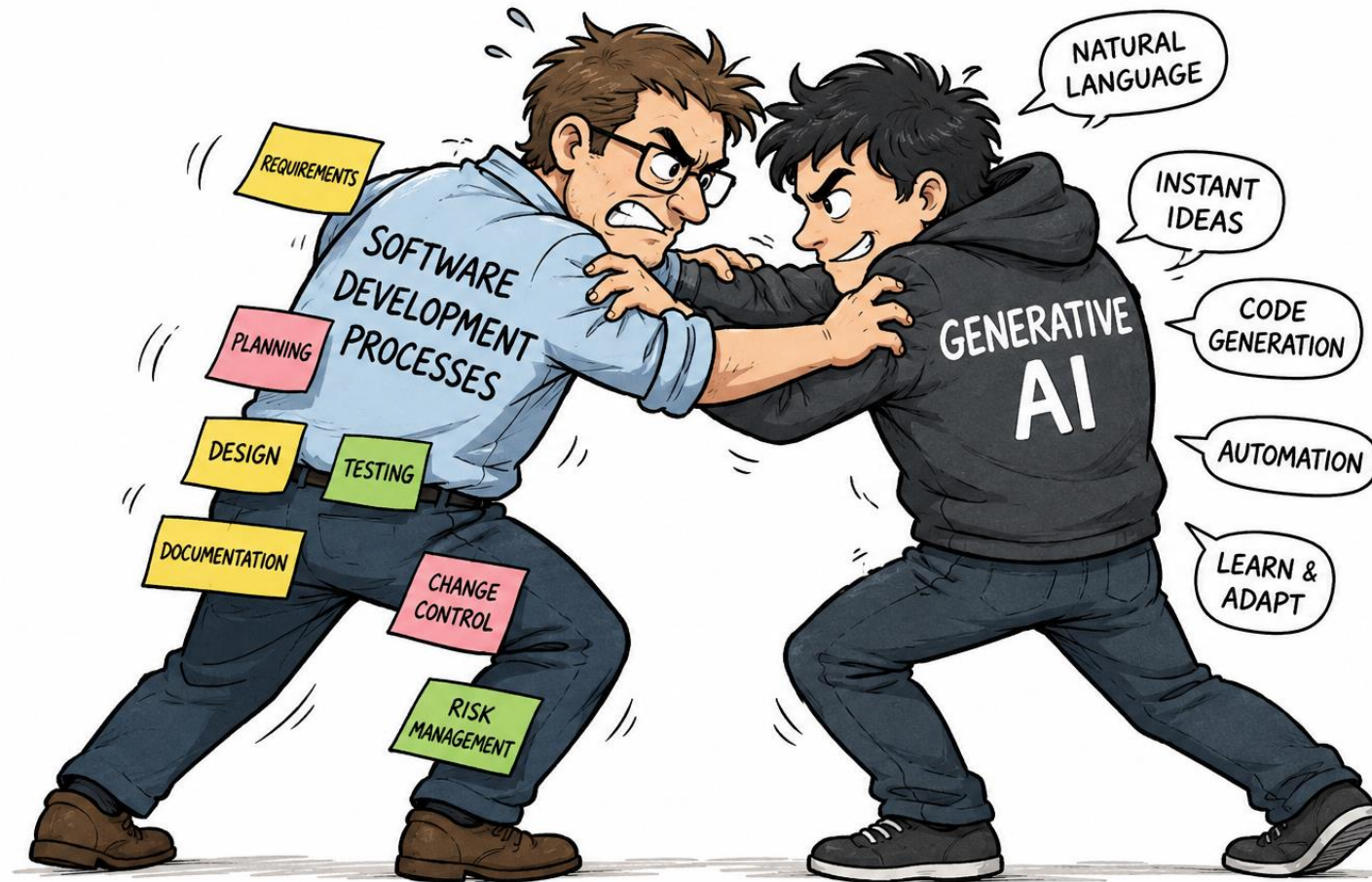
Agile

- Extreme Programming
- Kanban
- Scrum



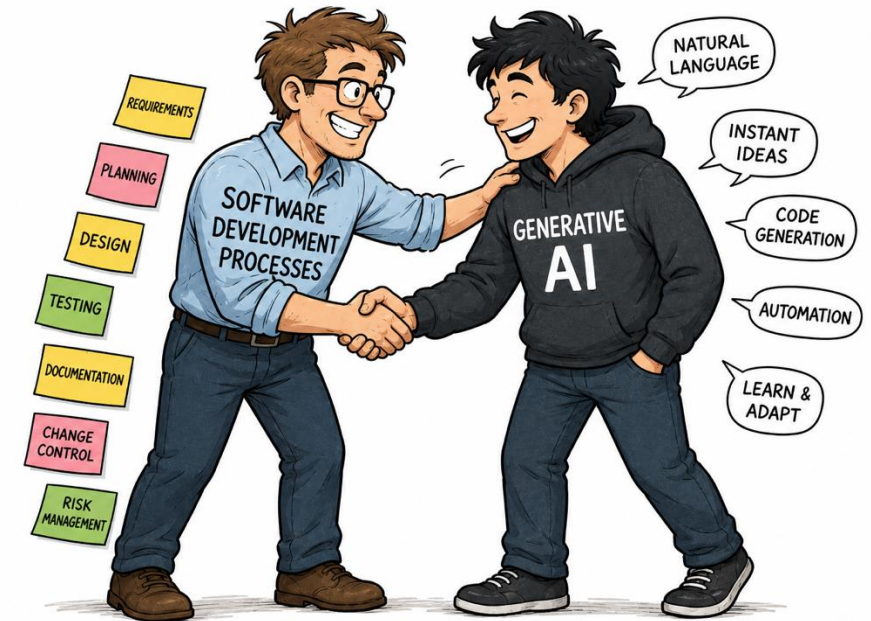
*Now for the elephant
in the room...*

Where Does AI Fit in Modern Software Delivery?



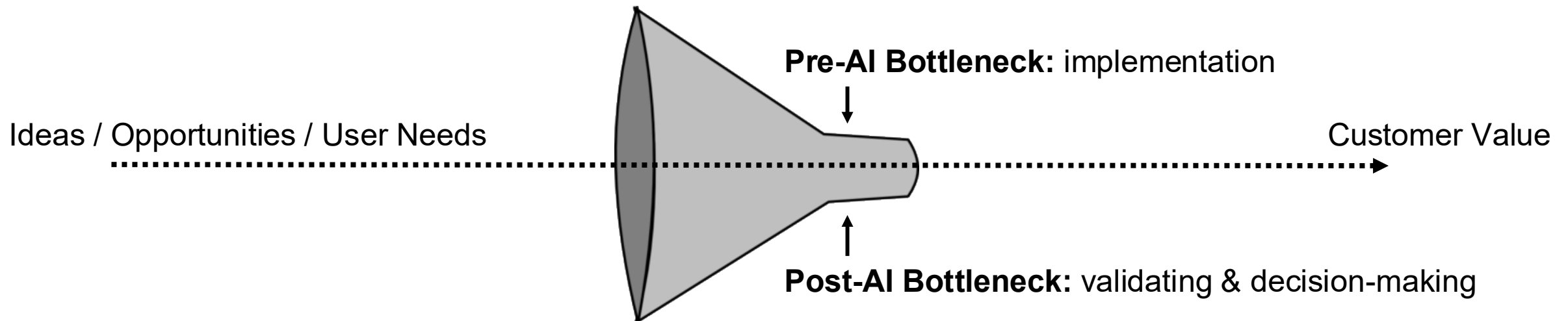
Where Does AI Fit in Modern Software Delivery?

- These processes were shaped before AI could write code — so what changes?
- AI and agents now write much of the code — but people still **direct, review, and stay accountable** for it
- Agile still frames the teamwork (scrum, sprints, backlogs) but the **practices adapt**: AI-assisted stories, estimation, refactoring (Weeks 2, 3, 11)



Building the Thing Right vs Building the Right Thing

- **Verification:** “building the thing right”: does it work? (tests, CI, code review — most of this course)
- **Validation:** “building the right thing”: is it worth building at all? Does it solve a real user problem? Is it what the user wanted?
- **AI’s impact:** when “building the thing right” gets cheap, “building the right thing” is what’s left — **the new bottleneck**



Group Exercise

Google Drive Exercise

In groups, copy and complete the provided template

Be ready to share and discuss with us!

Traditional

- Waterfall
- RUP

Agile

- Extreme Programming
- Kanban
- Scrum

Reminder – *Reflections*

- Please start filling in your weekly reflections now
- Complete them by two days after this class so that we have time to read them before we next meet 😊
- <http://smu.sg/is212-reflections-2026>

